

AIA Engineer - Vibe Coding Evaluation Project

Overview

This evaluation project assesses internal candidates for the AI Acceleration Engineer role. You are given a realistic client problem statement and requirements. Your job is to build a working skeleton using AI-assisted coding tools (Claude Code) - just as you would on the AIA team.

Time box: 2 weeks. Candidates should submit deliverables within the 2 week window.

Problem Statement

Client Scenario

A mid-size CPG company wants to understand their sales performance across product categories and regions. They have historical transaction data and want:

1. **A clean, reliable data foundation** - Raw transaction data needs to be ingested, validated, and stored in a queryable format. The data has quality issues (nulls, inconsistent formats) that must be handled.
2. **Predictive intelligence** - The business team wants to forecast revenue based on product category, region, and seasonal patterns. They also want to experiment with AI-powered insights - for example, using an LLM to generate natural language summaries of data trends or answer questions about the data.
3. **An API that serves both predictions and insights** - Downstream applications and dashboards need programmatic access to predictions, data summaries, and AI-generated insights.
4. **A simple interface to interact with the system** - Business users want to see key metrics, request predictions, and ask questions about their data without writing code.
5. **Production-readiness** - The solution must be containerized, tested, and deployable via CI/CD. A project team will inherit this and take it to production.

Data Landscape

A mid-size CPG company will typically have several data feeds relevant to this problem. Treat the following as a realistic landscape; design your schema and ingestion to fit it.

Sales transactions - timestamped purchase events with SKU, quantity, price, region/store. In a real CPG environment these flow from multiple POS systems and online channels, with the typical real-world wrinkles: schema drift between sources, late-arriving records, deduplication after retries, currency or unit normalization.

Product master / catalog - SKU metadata such as category, brand, package size, list price, launch date. Slow-changing dimension.

Regional / store reference - store-to-region mapping and demographic context for regions; possibly seasonal calendar or weather overlays where useful.

Optional secondary sources for richer signals - marketing campaign exposure, promo windows, competitive pricing snapshots; anything that helps explain demand patterns beyond seasonality.

Realistic data-quality issues are part of the problem.

Requirements

Build a working skeleton that addresses the problem statement above. The requirements are deliberately broad - how you interpret them, what trade-offs you make, and how you scope is part of the evaluation.

Functional requirements:

- Data flows end-to-end: ingestion through to a user-facing interface
- Predictions are based on actual data patterns, not hardcoded values
- There is an AI/LLM-powered component (e.g., natural language data summaries, question-answering over the data, trend analysis in plain English, or similar). This does not need to be sophisticated - using an LLM API to generate a paragraph summarizing query results counts. The point is demonstrating you can integrate AI capabilities, not build a production AI system.
- The system can be started and used by someone who has never seen the code before
- Tests exist and pass

Non-functional requirements:

- Containerized (Docker)
- CI pipeline defined (GitHub Actions or equivalent)
- Architecture decisions documented (at least 1 ADR)
- Clear extension points documented for the project team

What is NOT required:

- Deep ML sophistication - a linear regression that works beats a neural network that doesn't
- Production-grade infrastructure - a Dockerfile that runs is the bar

- Maximum code - a small, coherent system beats a large, disconnected one
- Single-domain depth - do NOT spend 4 hours on a perfect pipeline and ignore everything else

Technology Choice

Use any language and framework (Claude, OpenAI, or a local model), the choice is yours.

What to Submit

Candidates submit three deliverables. All three are scored after submission against a published rubric.

1. Git Repository

A repository that another engineer can clone, run, and continue extending. We're looking at it as if we were the project team inheriting this skeleton from you. Should include:

Working source across all functional areas of the problem
Tests
Setup and run instructions
Containerized deployment with a CI pipeline

The git history is part of the submission - we want to see how you worked with AI tools, not just what they produced.

2. Video Demo (5-10 minutes)

A short video walking through your project. Imagine the audience is the project team that will inherit this skeleton. Cover:

1. **Architecture overview - your design decisions and how things connect**
2. **Live demo - system working end-to-end**
3. **AI tool usage - at least one example of overriding or rewriting AI output**
4. **What you'd build next**

3. Methodology Spreadsheet (.xlsx)

Submit a spreadsheet documenting your vibe coding approach. This gives us insight into how you work with AI tools, not just what you built. Use the following structure:

Sheet	What to Include
Time Log	For each time block: duration, area worked on, AI tool used, what AI generated, what you changed/overrode
Tool Usage Summary	Total time, time split (planning/coding/testing/docs), primary AI tool, % AI-generated code, % modified, biggest AI win and miss
Decision Log	For each key decision: options considered, why you chose it, AI's suggestion, whether you followed AI

Logistics

This is a company-wide internal posting. Many people will apply. The entire process is **offline** - no interviews, no scheduling, no live sessions. Candidates complete the project on their own time and submit asynchronously.

How It Works

1. Internal posting goes out with this brief attached
2. Interested candidates self-select and complete the project within 2 weeks of applying

What Candidates Submit

1. **Git repository** - push final code
2. **Video demo** (5-10 minutes) - upload to the shared drive link provided
3. **Methodology spreadsheet** (.xlsx) - upload alongside the video